

Programación de Sistemas

El lenguaje IMP: descripción informal

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

IMP es un lenguaje imperativo pequeño. Fue diseñado para estudiar con claridad cómo se ejecuta y qué significa un programa, sin la cantidad de construcciones de un lenguaje de propósito general. Su ejecución modifica un **estado**: una asociación entre nombres y valores enteros. Para entender cómo se escribe un programa, primero distinguiremos sus piezas.

Expresiones aritméticas

Una **expresión aritmética** representa un cálculo cuyo resultado es un entero. Puede contener números, identificadores, suma +, resta -, multiplicación * y paréntesis. Evaluarla consulta los valores necesarios y calcula un resultado, pero no cambia por sí sola el estado.

- 20: representa directamente el entero veinte.
- x2: obtiene el entero almacenado en la ubicación x2.
- x2 + 1: suma uno al contenido actual de x2.
- X * (Y - 2): resta dos al contenido de Y y multiplica el resultado por el contenido de X.

Expresiones booleanas

Una **expresión booleana** representa una condición cuyo resultado es **true** o **false**. Puede usar esos dos valores, comparar expresiones aritméticas con igualdad = o menor o igual <=, y combinar condiciones mediante negación !, conjunción && o disyunción ||. Su resultado permite decidir qué comando ejecutar o si debe continuar una repetición.

- true: condición siempre verdadera.
- x2 <= 20: verdadera cuando el contenido de x2 no supera veinte.
- X = Y: compara los enteros almacenados en X y Y.
- !(X = 0) && X <= 10: exige que X sea distinto de cero y no sea mayor que diez.

Identificadores y ubicaciones

Un **identificador** es el nombre que aparece en el programa para referirse a una ubicación del estado. En el IMP básico del curso tiene una o más letras y puede terminar con cero o más dígitos. X, total y x20 son identificadores válidos; 20x, x2y y x_1 no cumplen esa forma.

El nombre y su contenido no son lo mismo. En x2 + 1, el identificador permite *leer* el entero almacenado; en x2 := 0, indica qué ubicación debe *actualizarse*. Las ubicaciones de IMP no se declaran antes de usarse.

Comandos

Un **comando** indica una acción o controla el orden en que se realizan otras acciones:

- `skip` no modifica el estado.
- La asignación `X := x2 + 1` evalúa la expresión aritmética de la derecha y guarda su resultado en `X`.
- La secuencia `X := 0; Y := X + 1` ejecuta primero el comando anterior al punto y coma y después el siguiente.
- `if X <= 3 then Y := 1 else Y := 0` evalúa una expresión booleana y elige uno de dos comandos.
- `while X <= 3 do X := X + 1` repite un comando mientras su expresión booleana produzca `true`.

Cómo se combinan en un programa

Un programa de IMP es un comando, posiblemente construido al poner en secuencia, condicionar o repetir otros comandos. Las expresiones aritméticas suministran enteros; las expresiones booleanas toman decisiones; los identificadores enlazan esas expresiones con el estado; y los comandos consultan o modifican dicho estado.

Ejemplo 1. Cálculo y secuencia de asignaciones

```
X := 2 + 3;  
Y := X * 2
```

La primera expresión aritmética produce 5 y la asignación lo guarda en `X`. Después del punto y coma, el segundo comando lee ese valor, lo multiplica por dos y guarda 10 en `Y`. El estado final contiene `X = 5` y `Y = 10`.

Ejemplo 2. Elegir mediante una condición

```
X := 4;  
if X <= 3 then Y := 1 else Y := 0
```

Después de guardar 4 en `X`, la expresión booleana `X <= 3` resulta falsa. Por ello se ejecuta el comando posterior a `else` y se guarda 0 en `Y`.

Ejemplo 3. Repetir con `while`

```
X := 0;  
while X <= 3 do X := X + 1
```

El primer comando establece el valor inicial. Después, el ciclo comprueba `X <= 3` y aumenta `X` en uno mientras la condición sea verdadera. Los valores producidos por las asignaciones del ciclo son 1, 2, 3 y 4; con `X = 4` la condición es falsa y termina la ejecución.

Ejemplo 4. Combinar condiciones booleanas

```
X := 5;  
if !(X = 0) && X <= 10 then Y := X - 1 else Y := 0
```

La condición exige simultáneamente que `X` sea distinto de cero y que no sea mayor que diez. Ambas partes son verdaderas cuando `X = 5`; se ejecuta entonces la primera rama y `Y` termina con el valor 4.

Ejemplo 5. Un comando que no modifica el estado

```
X := 1;  
if X = 1 then skip else X := 0
```

La comparación resulta verdadera y selecciona `skip`. Ese comando no cambia ninguna ubicación, por lo que el programa termina conservando 1 en `X`. La asignación de la rama `else` no se ejecuta.

Esta descripción presenta las piezas y su función, pero no fija todavía una gramática formal ni explica cómo analizarlas. Esas tareas se desarrollan después al estudiar el análisis léxico y sintáctico.